Integration Test Report

Deliverable 3, Campus Ride-Pooling and Split-Fare System

Prepared by Mainak Sarkar Jira CS455-110, CS455-111, CS455-112, CS455-114 Run of 2026-10-28

Prithviraj Ghosh (230794) Mainak Sarkar (230619) Khushi Ranawat (230562) Mahir Jain (230618) Chaitanya Nitawe (230315)

Result. 66 of 66 integration tests passed in 10 files (7 of them regression tests for defects this suite found), against a real PostgreSQL 16 database, locally and in GitHub Actions. Together with the 41 adversarial cases (separate report) the run had 107/107 passing in 36.2 s of test time. Two defects were found and fixed with regression tests (Section 6).

1. Purpose and scope

The unit suite (about 1,500 tests, reported separately) checks each module in isolation, with the database and the session mocked. It cannot see a query that PostgreSQL rejects, a constraint or trigger that refuses a write, or two modules disagreeing about the rows they share. This suite closes that gap: every test calls the *real* API route handlers, which run their real SQL against a real database, and checks both the HTTP answer and the rows written.

In scope: the core transactional workflow of the D2 demonstration (offer, search, request, decide, lock, pickup, trip, completion, rating, complaint) and the safety and operations flows around it (SOS and incident handling, warnings, suspensions, payment reminders). Out of scope, and covered elsewhere: rendered pages and the browser (system tests), the 50-way last-seat experiment (concurrency report), access control and IDOR (security report), latency (performance report).

2. Test harness

2.1 What runs for real, and what is replaced

Real	Route handlers, session lookup (`getCurrentUser`), the same-origin check and database rate limit (`guard`), the admin role check, every SQL query, constraint, trigger and transaction, the Gemini fallback path.
Replaced	`next/headers` (`cookies()`, `headers()`) and `next/server` (`after()`): Next.js answers them only inside its own request handling. Small shims read the current test request from `AsyncLocalStorage`; work passed to `after()` runs right after the handler answers, as Next.js runs it after the response.
Switched off	Outgoing email and Gemini (empty keys), so no test calls an external service; the app's documented fallbacks run instead.

Routes are called in-process with `callRoute(handler, {method, body, params, as, origin, ip})`, which builds the `Request`, sets the session cookie of the caller, a same-origin or cross-site `Origin`, and a fresh client address per call (so per-address rate limits are only exercised on purpose). In-process calls need no production build, run the whole suite in under a minute, and give full stack traces; the deployed stack end to end is the system tests' job.

2.2 Test data and time

- **Accounts and sessions** are written with SQL (`test/seed/accounts.ts`), because registering through the app sends a real verification email. Only a token's SHA-256 is stored, exactly as the app stores it. The same seed is shared with the Playwright system tests.
- **Everything else goes through the API**: rides are offered, seats requested and accepted through the real routes, so the rows are those the app itself writes.
- **Time** is the one thing simulated: `moveDeparture(ride, minutes)` moves a ride's departure, and the departure and pickup times its riders agreed to, by the same amount, as if the clock had moved on (the move `setup.md` documents for trying a trip).

2.3 Isolation and safety

Each run drops and recreates the test database and applies `db/schema.sql`, the file production migrations apply, so the schema is also checked on an empty database every run. Before every test all tables are truncated and the rows the schema itself inserts (campus places, hubs, vehicle types, tags) are put back, so every test starts from a freshly migrated database. The harness refuses any database whose name does not end in `_test`, before connecting and again in each worker, so it can never empty the application database.

3. Environment and how to rerun

Runtime	Node.js 24, Next.js 16.3, Vitest 5.0.3, node-postgres 8
Database	PostgreSQL 16: Docker Compose locally (`127.0.0.1:5433`), a service container in CI
CI	Job *Integration tests (PostgreSQL)* in `.github/workflows/ci.yml`, on every PR and push to master; latest run on the stacked PRs: run 40734814513 (pass)

```
docker compose up -d db                                                  (from the repository root)
TEST_DATABASE_URL=postgres://crp:<password>@127.0.0.1:5433/crp_test      (in apps/web/.env.local)
npm run test:integration                                                 (from apps/web)
```

The tables in this report: `Deliverables/D3/test-run/README.md`.

4. Results

Table 1. Integration test files, from the recorded run (`test-run/integration-results.json`).

File (under `apps/web/test/integration/`)	Tests	Passed	Time (s)
`harness.int.test.ts`	6	6	1.3
`admin/complaints-and-enforcement.int.test.ts`	9	9	4.4
`bugs/idempotency-key-reuse.int.test.ts`	3	3	0.9
`bugs/timestamp-parameters.int.test.ts`	4	4	1.0
`rides/join-requests.int.test.ts`	9	9	2.9
`rides/manage-ride.int.test.ts`	5	5	1.6
`rides/offer-and-search.int.test.ts`	7	7	1.8
`rides/seats-and-lock.int.test.ts`	7	7	2.2
`trip/sos.int.test.ts`	7	7	3.2
`trip/trip-flow.int.test.ts`	9	9	3.4
Total	**66**	**66**	

Table 2. Code coverage of the integration run alone (%), from `test-run/integration-coverage-summary.json`. Pages and components are not reachable through API routes, so the whole-`src` figure is low by design; the unit suite carries the 80% gate (NFR-RD-17).

Area	Statements	Branches	Functions	Lines
API route handlers (`src/app/api`)	36.0	29.8	43.9	40.2
Server library (`src/lib`)	34.4	26.1	37.0	36.8
All of `src` (pages and components included)	22.8	17.4	19.4	24.2

5. Requirements covered

Each row names the file that exercises the requirement; every case is listed in the appendix.

Requirement	File	Checked
FR-RO-01 to 03, P-08: offer a ride	`rides/offer-and-search`	stored ride and owner seat; field errors; overlap and two-hour conflict
FR-RD-03, FR-RD-08.1: search	`rides/offer-and-search`	seats left, estimated share, hidden own, joined, locked, full and filtered rides
FR-RD-06, FR-RD-06.9: request a seat, idempotency	`rides/join-requests`, `bugs/idempotency-key-reuse`	snapshot, history, key; one request per retried key; limit of three

Requirement	File	Checked
FR-RO-05: owner decides	`rides/join-requests`	accept, reject, share notices; only the owner decides
FR-RD-06.10, FR-RD-07: withdraw and leave	`rides/join-requests`, `rides/seats-and-lock`	seat freed before lock; late leave keeps the locked share
FR-RO-07, P-06: lock	`rides/seats-and-lock`	shares fixed owner first (334/333/333), waiting requests expired, lock at time
FR-RO-04: edit and cancel	`rides/manage-ride`	confirmation for accepted riders, reconfirmation after a moved departure, cancel until the window ends
FR-RO-09, FR-RD-10: trip states	`trip/trip-flow`	lock before pickup, no skipping, check-in, arrivals, start trip, completion
FR-RO-10, P-19: no-show	`trip/trip-flow`	refused inside the grace period, recorded after; share spread over the rest
FR-RO-09.4: final shares	`trip/trip-flow`	written once, sum equals the fare
FR-RD-15, FR-RO-13.2: SOS	`trip/sos`	one alert per person; no trace for the pool; snapshot, location, false alarm
CS455-65, SYS-NFR-08: incident handling	`trip/sos`	acknowledge, close with a note, every attempt audited; students refused
FR-RD-13, SYS-FR-46: complaints	`admin/complaints-and-enforcement`	window from completion or own no-show; one per person; Gemini fallback recorded; admin review with version check
FR-RD-12, P-16: ratings	`admin/complaints-and-enforcement`	once per rater; never self or a no-show
OA-FR-21 to 28: warnings and suspensions	`admin/complaints-and-enforcement`	warning blocks until acknowledged; suspension blocks offering and joining until lifted; database trigger
FR-RD-11: overdue payment, reminders	`bugs/timestamp-parameters`	request blocked after 7 days due; 24 h then 72 h reminder, once each

6. Defects found

#	Defect	Found by	Fix and test
1	**Every seat request answered 500 on a real database**, and so did the payment reminder scheduler. `completed_at < $2 - interval '7 days'` with a JavaScript `Date`: node-postgres sends it untyped, PostgreSQL reads `$2 - interval` as interval minus interval, and no operator compares a timestamp with an interval. Introduced on 14 Oct with the settlement feature; invisible to the mocked unit tests. Fixed by casting the parameter.	This harness, first run	PR #84; `bugs/timestamp-parameters` (4 tests, all fail without the fix)
2	**An idempotency key sent again for another ride or pickup point replayed the first request**: the rider was told "already submitted" and nothing was created for the ride asked for, or a changed pickup was dropped. Now refused with 409 `idempotency_key_reused`.	Adversarial suite	PR #87; `bugs/idempotency-key-reuse` (2 of 3 fail without the fix)

Both are tracked as Jira Bug issues with the `d3-defect` label and listed in the team's bug history.

7. Limitations

- **No browser and no Socket.IO.** Pages, the live chat socket and the in-process timers in `server.mjs` and `instrumentation.ts` are not exercised here; the Playwright system tests cover them on the Docker stack.
- **Gemini's success path is not called**, only its fallback: tests must not depend on an external service or spend its quota. The AI paths are covered by unit tests with recorded responses.
- **Time is simulated by moving data**, not the clock. The moved fields are the ones the app reads for each rule; a rule that read another time field would not see the move.
- **One unexplained failure.** In one of nine consecutive full runs the payment reminder test failed once; it passed in the eight runs after it and in six isolated runs, and in every CI run. It is being watched rather than hidden.

A. Every integration case

File	Case (describe block: test)	Result
`harness`	integration harness: offers a ride through the real API and reads it back from the database	Pass
`harness`	integration harness: runs the real sign-in and origin checks	Pass
`harness`	integration harness: reads the role from the database for admin routes	Pass
`harness`	integration harness: moves a ride into its lock window for trip tests	Pass
`harness`	a clean database for every test: writes a ride	Pass
`harness`	a clean database for every test: no longer sees it, and still has the schema reference data	Pass
`complaints-and-enforcement`	complaints (FR-RD-13): opens on completion, files once per person, and falls back to manual review without Gemini	Pass
`complaints-and-enforcement`	complaints (FR-RD-13): refuses before the trip, outsiders, oneself and a too-short description	Pass
`complaints-and-enforcement`	complaints (FR-RD-13): lets a rider marked as a no-show dispute it before the ride is completed	Pass
`complaints-and-enforcement`	complaints (FR-RD-13): is reviewed by an admin with a version check, and every step is audited	Pass
`complaints-and-enforcement`	ratings (FR-RD-12, FR-RO-12): opens on completion, once per rater, never for oneself	Pass
`complaints-and-enforcement`	ratings (FR-RD-12, FR-RO-12): is closed before completion and never covers a no-show	Pass
`complaints-and-enforcement`	warnings and suspensions (OA-FR-21 to OA-FR-28): blocks a warned rider until they acknowledge the warning	Pass
`complaints-and-enforcement`	warnings and suspensions (OA-FR-21 to OA-FR-28): stops a suspended rider offering or joining rides until an admin lifts it	Pass
`complaints-and-enforcement`	warnings and suspensions (OA-FR-21 to OA-FR-28): allows enforcement by admins only, in the API and in the database	Pass
`idempotency-key-reuse`	an idempotency key sent again with other details: is refused for another ride, and creates nothing there	Pass
`idempotency-key-reuse`	an idempotency key sent again with other details: is refused for another pickup point, keeping the first request as it was	Pass
`idempotency-key-reuse`	an idempotency key sent again with other details: still replays the same request sent again unchanged	Pass
`timestamp-parameters`	timestamp parameters compared with an interval: lets a rider with no unpaid shares request a seat	Pass
`timestamp-parameters`	timestamp parameters compared with an interval: still refuses a rider whose share has been due for more than seven days	Pass
`timestamp-parameters`	timestamp parameters compared with an interval: allows a share that has been due for less than seven days	Pass
`timestamp-parameters`	timestamp parameters compared with an interval: sends each payment reminder once, 24 hours and then 72 hours after completion	Pass
`join-requests`	requesting a seat (FR-RD-06): records a pending request with its snapshot, history and idempotency key, and tells the owner	Pass
`join-requests`	requesting a seat (FR-RD-06): answers a retried request with the same idempotency key once, without a second request (FR-RD-06.9)	Pass
`join-requests`	requesting a seat (FR-RD-06): refuses the owner, an unknown pickup point and a fourth waiting request	Pass
`join-requests`	requesting a seat (FR-RD-06): shows the owner the waiting requests and the rider their own history	Pass

File	Case (describe block: test)	Result
join-requests	the owner deciding (FR-RO-05): accepting gives the rider a seat, updates everyone's share and records why	Pass
join-requests	the owner deciding (FR-RO-05): rejecting leaves the seat free and tells the rider	Pass
join-requests	the owner deciding (FR-RO-05): lets only the owner decide, and only the rider withdraw	Pass
join-requests	the rider changing their mind (FR-RD-06.10, FR-RD-07): withdraws a waiting request once, and the owner is told	Pass
join-requests	the rider changing their mind (FR-RD-06.10, FR-RD-07): leaves before lock with nothing owed, freeing the seat for others	Pass
manage-ride	cancelling a ride (FR-RO-04.2): cancels every request and seat claim and tells each rider they owe nothing	Pass
manage-ride	cancelling a ride (FR-RO-04.2): refuses anyone but the owner, and a ride whose window is over	Pass
manage-ride	editing a ride (FR-RO-04.1): asks the owner to confirm a change that affects accepted riders, then tells them	Pass
manage-ride	editing a ride (FR-RO-04.1): sends a waiting request back to the rider when the departure moves more than 10 minutes	Pass
manage-ride	editing a ride (FR-RO-04.1): refuses edits that break the ride's rules	Pass
offer-and-search	offering a ride (FR-RO-01 to FR-RO-03): stores the ride with the vehicle capacity and gives the owner the first seat	Pass
offer-and-search	offering a ride (FR-RO-01 to FR-RO-03): refuses an invalid offer field by field and stores nothing	Pass
offer-and-search	offering a ride (FR-RO-01 to FR-RO-03): refuses a ride that overlaps or departs within two hours of the owner's other ride	Pass
offer-and-search	searching for a ride (FR-RD-03, FR-RD-08.1): shows an open ride with its seats and the share a new rider would pay	Pass
offer-and-search	searching for a ride (FR-RD-03, FR-RD-08.1): never shows the owner their own ride, nor a rider a ride they are on	Pass
offer-and-search	searching for a ride (FR-RD-03, FR-RD-08.1): hides rides that are locked, full, or outside the filters	Pass
offer-and-search	searching for a ride (FR-RD-03, FR-RD-08.1): keeps a ride the viewer has asked to join visible with its request state	Pass
seats-and-lock	the last seat (FR-RO-05.3, NFR-RD-05): refuses a request when the ride is full, and an accept that would overfill it	Pass
seats-and-lock	the last seat (FR-RO-05.3, NFR-RD-05): withdraws a rider's other request within two hours once they get a seat (P-08)	Pass
seats-and-lock	locking the ride (FR-RO-07, P-06): fixes every share when the owner locks early, owner first, and closes waiting requests	Pass
seats-and-lock	locking the ride (FR-RO-07, P-06): lets only the owner lock	Pass
seats-and-lock	locking the ride (FR-RO-07, P-06): locks by itself an hour before departure, on the next request	Pass
seats-and-lock	leaving after lock (FR-RD-07.2, P-10): asks the rider to confirm, then keeps their locked share payable	Pass
seats-and-lock	leaving after lock (FR-RD-07.2, P-10): refuses a malformed confirmation flag	Pass
sos	raising an SOS: creates one alert per person, however often they tap, and tells nobody in the pool	Pass
sos	raising an SOS: keeps the mobile number and a snapshot of the trip for the admins	Pass
sos	raising an SOS: is only for people on the ride while the trip is under way	Pass
sos	raising an SOS: lets only the person who raised it add a location or call it a false alarm	Pass
sos	admins handling an SOS (CS455-65): shows the alert until acknowledged, then closes it with a note, auditing each step	Pass
sos	admins handling an SOS (CS455-65): keeps students out of the incident routes	Pass
sos	admins handling an SOS (CS455-65): keeps the original alert unchangeable in the database	Pass
trip-flow	starting pickup: waits for the lock, and only the owner can start it	Pass
trip-flow	starting pickup: cannot skip pickup, and cannot start once the departure window is over	Pass
trip-flow	starting pickup: rejects an unknown step	Pass
trip-flow	during pickup: lets a rider check in and the owner mark them arrived or undo it	Pass
trip-flow	during pickup: refuses a no-show until five minutes after the rider's pickup time (P-19)	Pass
trip-flow	during pickup: records a no-show after the grace period: off the ride, owes nothing, told how to dispute	Pass

File	Case (describe block: test)	Result
`trip-flow`	the trip and completion (FR-RO-09.4): stores the final pool and shares: the no-show's share is spread over the others	Pass
`trip-flow`	the trip and completion (FR-RO-09.4): counts unmarked riders as on board when the trip starts	Pass
`trip-flow`	the trip and completion (FR-RO-09.4): answers the live trip stamp to people on the ride only	Pass